

MatchCake: A Python Simulator for Non-Interacting Fermionic Quantum Circuits with Machine Learning Applications

J       Gince ¹

¹ Institut Quantique & D  partement de Physique, Universit   de Sherbrooke, Sherbrooke, QC J1K 2R1, Canada

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 20 February 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

We introduce MatchCake, a new Python package that implements a simulator of matchgate quantum circuits (also known as matchcircuits). Matchgates form a restricted class of unitaries that preserves parity and act on nearest-neighbor qubits. These circuits are equivalent to a hamiltonian evolution of non-interacting Majorana fermions, which makes them classically simulable in polynomial time while retaining rich quantum structure. MatchCake provides a practical framework for exploring this class of circuits within PennyLane ([Bergholm & et.al, 2022](#)) and PyTorch ([Ansel et al., 2024](#)), enabling research on classically simulable quantum machine learning models with automatic differentiation. We demonstrate its capabilities through applications in quantum kernel methods, including classification ([Gince et al., 2024](#)), regression, and beyond.

Statement of need

In quantum computing, quantum circuits provide a formal framework for representing the discrete time evolution of a quantum state under a sequence of operations known as quantum gates. They constitute the backbone of quantum algorithms, quantum system simulations, and broader quantum information processing tasks. They are therefore central to the research and development of methods that exploit quantum resources such as superposition, entanglement, and interference. However, since quantum hardware is still under active development, it is crucial to have efficient tools for simulating and analyzing quantum circuits on classical computers. Indeed, the classical simulation of quantum circuits is essential for the study of quantum systems, the design of quantum algorithms, and the development of quantum computers themselves. Moreover, the study of classically simulable circuit families is itself a key avenue for understanding the boundary between classical and quantum computational power. As a result, a large number of quantum circuit simulators have been developed to address this need. In general, the state space of an N -qubit system has dimension $\mathcal{O}(2^N)$, so an explicit representation of a generic quantum state requires exponentially many parameters, which leads most simulation approaches to scale exponentially in the number of qubits. Nonetheless, efficient simulation becomes possible for restricted families of circuits whose states or evolutions admit compact, polynomial-size representations.

Quantum circuit simulators are differentiated by the classes of circuits they can simulate efficiently. For example, Stim ([Gidney, 2021](#)) is designed for stabilizer circuits and achieves polynomial complexity, $\mathcal{O}(N^2)$ where N is the system size, when simulating circuits composed of Clifford gates. On the other hand, quimb ([Gray, 2018](#)) leverages tensor network methods ([Biamonte & Bergholm, 2017](#)), such as matrix product states (MPS), to efficiently simulate

circuits with limited entanglement at a cost of $\mathcal{O}(N\chi^3)$, where χ is the bond dimension, which is a proxy for the amount of entanglement in the system.

At a higher level of abstraction, PennyLane (Bergholm & et.al, 2022) provides a widely used Python framework for quantum circuit simulation and quantum machine learning. PennyLane enables the definition, execution, and differentiation of quantum circuits across a variety of backends, making it particularly well suited for hybrid quantum–classical algorithms and learning-based applications. While PennyLane supports multiple simulators and hardware interfaces, efficient native support for matchgate-based circuits is currently lacking.

Specifically, PennyLane primarily relies on universal state-vector simulators (SVS), which, although capable of simulating arbitrary quantum circuits, suffer from an exponential scaling in the number of qubits, thereby are fundamentally not scalable. PennyLane also provides wrappers around specialized simulators such as Stim and quimb, enabling more efficient simulation for certain restricted classes of circuits. However, none of these backends target the subclass of quantum circuits composed of matchgates (matchcircuits (Valiant, 2001)). Consequently, users interested in simulating matchgate-based circuits are forced to rely on exponentially scaling simulators, despite the fact that such circuits theoretically admit polynomial-time $\mathcal{O}(N^3)$ classical simulation (Bravyi, 2008; Brod, 2016; Hebenstreit et al., 2020).

Unlike tensor-network-based methods, whose efficiency relies on restricting entanglement growth, the efficient simulability of matchcircuits arises from an underlying free-fermionic (Gaussian) structure, allowing them to support arbitrarily large entanglement while remaining tractable. In contrast to Clifford circuits, which are efficiently simulable due to a discrete and non-parametrized gate set, matchcircuits are composed of matchgates (Jozsa & Miyake, 2008), a continuously parametrized class of two-qubit unitary gates corresponding to free-fermionic operations. Despite their theoretical importance and favorable resource trade-offs, there is currently no clear, dedicated tool for their simulation within general-purpose quantum software frameworks.

On a technical side, a two-qubit matchgate written in the computational basis

$$M(A, W) = \begin{bmatrix} a & 0 & 0 & b \\ 0 & w & x & 0 \\ 0 & y & z & 0 \\ c & 0 & 0 & d \end{bmatrix}, \quad (1)$$

is a parity-preserving nearest-neighbour unitary gate defined by two submatrices

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, W = \begin{bmatrix} w & x \\ y & z \end{bmatrix}, \quad (2)$$

subject to the constraint $\det(A) = \det(W)$. Indeed, when restricted to such matchcircuits on a linear register of qubits, the system’s dynamics become equivalent to those of non-interacting Majorana fermions (Dai et al., 2015; Nielsen & others, 2005).

Recent theoretical work has further clarified their position within the broader landscape of efficiently simulable quantum circuits. In particular, (Projansky et al., 2025) established a formal connection between matchcircuits and Clifford circuits through fermionic Gaussian operations, highlighting matchcircuits as a natural generalization that interpolates between stabilizer dynamics and more general fermionic evolutions.

Beyond their theoretical interest, matchcircuits and their extensions have practical relevance for quantum simulation. Notably, (Mocherla et al., 2023) showed that augmenting matchcircuits with additional ZZ interactions enables the efficient simulation of an interaction-limited

80 Fermi–Hubbard system. This extension significantly broadens the class of physically relevant
 81 problems accessible to matchgate-based simulation techniques, encompassing applications in
 82 quantum thermodynamics (Chapman & Miyake, 2018), quantum machine learning (Gince et
 83 al., 2024; Laar, 2022), and many-body quantum system simulation (Mocherla et al., 2023).

84 MatchCake is then designed to fill the gap in efficient simulation tools for matchcircuits. By
 85 providing a dedicated simulator integrated with PennyLane and PyTorch (Ansel et al., 2024),
 86 MatchCake enables researchers to explore the capabilities of matchcircuits in quantum machine
 87 learning and other applications. Its automatic differentiation support facilitates the development
 88 of learning-based models that leverage the unique properties of matchgate dynamics, opening
 89 new avenues for research in classically simulable quantum algorithms.

90 Research Impact Statement

91 MatchCake has already demonstrated its usefulness in the context of the paper “Fermionic
 92 Machine Learning” (Gince et al., 2024), whose goal was to benchmark different kernel-based
 93 machine learning models for data classification. By using the present package, the authors
 94 were able to demonstrate the performance of free-fermionic models on a large number of
 95 qubits, an analysis that would not have been feasible with a state-vector simulator (SVS).

96 In that work, the authors emphasize the necessity of relying on such classically simulable
 97 models when benchmarking quantum machine learning (QML) approaches, in order to assess
 98 whether the claimed quantum resources are truly required to achieve strong performance. This
 99 type of verification is essential in the current landscape, prior to making any claims of quantum
 100 advantage in QML. From another perspective, the authors also discuss a key advantage of
 101 free-fermionic models over other QML approaches: the absence of barren plateaus, a property
 102 of critical importance for the efficient optimization of machine learning models.

103 It therefore follows that MatchCake is an essential tool for the quantum simulation and
 104 quantum machine learning communities. It is not only valuable for fundamental research in
 105 quantum computing and for the development of QML models, but also plays a central role in
 106 validating the claimed advantages of such models.

107 Functionality

108 Using MatchCake, the user is expected to provide a matchcircuit C , which is a list of matchgates,
 109 the initial state $|\psi_{\text{initial}}\rangle$ of the quantum system, required to be a product state, and the
 110 observable $\mathcal{F}$ to be measured. The observable $\mathcal{F}$ must be a linear combination

$$\mathcal{F} = \sum_j \beta_j F_j \quad (3)$$

111 of observables F_j that can be efficiently decomposed into Gaussian Majorana observables which
 112 take the form

$$\begin{aligned} F_j \in & \{ |y\rangle \langle y| : y \in \{0, 1\}^N \} \\ & \cup \left\{ \bigcup_{i=0}^{N-1} \{ Z_i I_{i+1}, X_i X_{i+1}, Y_i Y_{i+1}, Y_i X_{i+1}, X_i Y_{i+1}, I_i Z_{i+1} \} \right\} \\ & \cup \left\{ \bigcup_{i < k=0}^{N-1} \{ Z_i Z_k \} \right\} \end{aligned} \quad (4)$$

where $|y\rangle$ is any computational-basis state, X_i , Y_i , and Z_i denote the Pauli matrices acting on qubits i of a linear register of N qubits, $i \in \{0, \dots, N-2\}$ labels nearest-neighbour sites, and $k \in \{1, \dots, N-1\}$ denotes an arbitrary qubit index. In return, MatchCake computes the expectation value

$$\text{Output} = \langle \psi_{\text{initial}} | C^\dagger \mathcal{F} C | \psi_{\text{initial}} \rangle \quad (5)$$

in time polynomial in N , the number of qubits in the system.

More specifically, the MatchCake algorithm presented in Figure 1 is divided into three distinct branches, depending on the observable $\mathcal{F}$ to be measured. Branch (A) handles projectors in the computational basis. Branch (B) treats the $Z_i I_{i+1}$, $I_i Z_{i+1}$, $Z_i Z_k$ Pauli words by expressing them as a linear combination of branch-(A) evaluations. Branch (C) addresses the remaining Pauli words.

Across the different stages of the algorithm, several intermediate procedures are employed:

1. the Majorana decomposition of the observable,
2. the use of a lookup table to obtain a special tensor $\mathcal{A}$ which represents the Wick's contraction (Wick, 1950) of a Majorana string (Brod, 2016),
3. the mapping of the matchgate circuit C to its corresponding transition matrix R_C (Brod, 2016),
4. the Jordan–Wigner transformation (Jordan & Wigner, 1928; Nielsen & others, 2005), and
5. the use of a Clifford subroutine based on PennyLane's stabilizer simulator (Bergholm & et.al, 2022) to obtain the final expectation value.

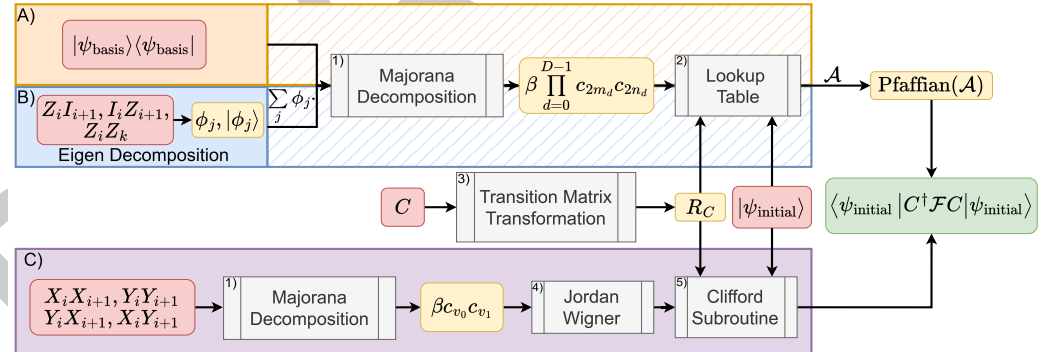


Figure 1: The MatchCake algorithm. Algorithm inputs are shown as red rounded rectangles, intermediate results as yellow rounded rectangles, and the final output as a green rounded rectangle; processing steps are represented by grey rectangles. The algorithm branches into three distinct paths depending on the observable $\mathcal{F}$ being measured: (A) a computational-basis projector, (B) a Pauli word diagonal in the computational basis, and (C) other specific Pauli words that can be decomposed into Gaussian Majorana observables. The coefficients β are real scalars arising from the Majorana decomposition.

Minimal Example - Quantum Circuit Simulation

In the following code block, we demonstrate a simple example on how to use MatchCake to create a matchcircuit and obtain the expectation value of a computational-basis projector after applying the circuit to an initial state.

```
import matchcake as mc
import pennylane as qml
import numpy as np
from pennylane.ops.qubit.observables import BasisStateProjector
```

```
# Create a Non-Interacting Fermionic Device with 4 qubits/wires
nif_device = mc.NonInteractingFermionicDevice(wires=4)
# Define the initial state as the all-zero computational basis state: |0000>
initial_state = np.zeros(len(nif_device.wires), dtype=int)

# Define a quantum circuit
def circuit(params, wires, initial_state):
    # Prepare the initial state
    qml.BasisState(initial_state, wires=wires)
    for i, even_wire in enumerate(wires[:-1:2]):
        idx = list(wires).index(even_wire)
        curr_wires = [wires[idx], wires[idx + 1]]
        # Apply the matchgate M(Rx(params), Rx(params))
        mc.operations.CompRxRx(params, wires=curr_wires)
        # Apply the matchgate M(Ry(params), Ry(params))
        mc.operations.CompRyRy(params, wires=curr_wires)
        # Apply the matchgate M(Rz(params), Rz(params))
        mc.operations.CompRzRz(params, wires=curr_wires)
    for i, odd_wire in enumerate(wires[1:-1:2]):
        idx = list(wires).index(odd_wire)
        mc.operations.fSWAP(wires=[wires[idx], wires[idx + 1]])
    projector: BasisStateProjector = qml.Projector(initial_state, wires=wires)
    return qml.expval(projector)

# Create a QNode
nif_qnode = qml.QNode(circuit, nif_device)
qml.draw_mpl(nif_qnode)(
    params=np.array([0.1, 0.2]),
    wires=nif_device.wires,
    initial_state=initial_state
)

# Evaluate the QNode
expval = nif_qnode(
    params=np.array([0.1, 0.2]),
    wires=nif_device.wires,
    initial_state=initial_state
)
print(f"Expectation value: {expval:.4f}")
```

137 Output: Expectation value: 0.9901

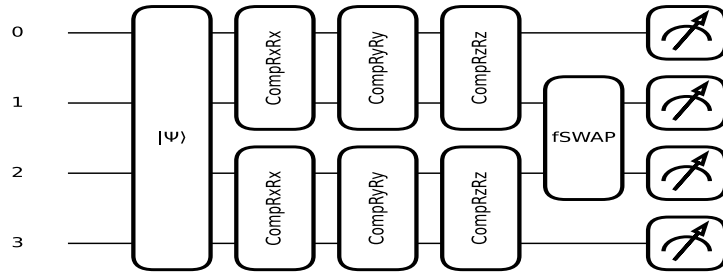


Figure 2: Quantum circuit generated by the circuit function.

138 Minimal Example - Quantum Machine Learning

139 In the following code block, we demonstrate a simple example on how to use MatchCake
140 to create quantum kernel-based classification models using the FermionicPQCKernel and
141 LinearNIFKernel classes. We then evaluate its performance on the iris dataset (Fisher, 1936)
142 using cross-validation with 20 random folds. The framework used in the MatchCake.ml is built
143 on top of Scikit-learn (Buitinck et al., 2013; Pedregosa et al., 2011), providing a familiar and
144 consistent interface for machine learning practitioners.

```
import matplotlib as mpl
from matchcake.ml import CrossValidation, CrossValidationVisualizer
from matchcake.ml.kernels import FermionicPQCKernel
from matchcake.ml.kernels.linear_nif_kernel import LinearNIFKernel
from matchcake.ml.visualisation import ClassificationVisualizer
from matplotlib import pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.svm import SVC

# Load the iris dataset
dataset = datasets.load_iris(as_frame=True)
X, y = dataset.data, dataset.target

# Define estimators using different kernels for comparison
n_qubits = 8
estimators = {
    "Fermionic PQC": Pipeline(
        [
            ("scaler", MinMaxScaler(feature_range=(0, 1))),
            # Example of kernel without alignment, which is the default behaviour
            ("kernel", FermionicPQCKernel(n_qubits=n_qubits, rotations="X,Z")),
            ("classifier", SVC(kernel="precomputed")),
        ]
    ),
    "Fermionic Linear": Pipeline(
        [
            ("scaler", MinMaxScaler(feature_range=(0, 1))),
            # Example of kernel with alignment, which improves the performance
```

```

        ("kernel", LinearNIFKernel(n_qubits=n_qubits, alignment=True)),
        ("classifier", SVC(kernel="precomputed")),
    ],
),
}

# Run cross-validation
cvo = CrossValidation(estimators, X, y).run()

# Create subplots for visualization
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))
# Visualize the classification boundaries for a selected model and fold
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
viz = ClassificationVisualizer(x=X, n_pts=25_000)
viz.plot_2d_decision_boundaries(
    model=estimators["Fermionic Linear"].fit(x_train, y_train),
    title="",
    legend_labels=dataset.target_names,
    y=y,
    fig=fig,
    ax=axes[0],
    legend_loc="lower right",
)
# Visualize cross-validation results with violin plots
cv_viz = CrossValidationVisualizer(cvo)
cv_viz.plot(
    ax=axes[1],
    score_name="Accuracy",
    estimator_name_key="Kernel",
    score_name_map={"train_score": "Train", "test_score": "Test"},
    palette="pastel",
)

```

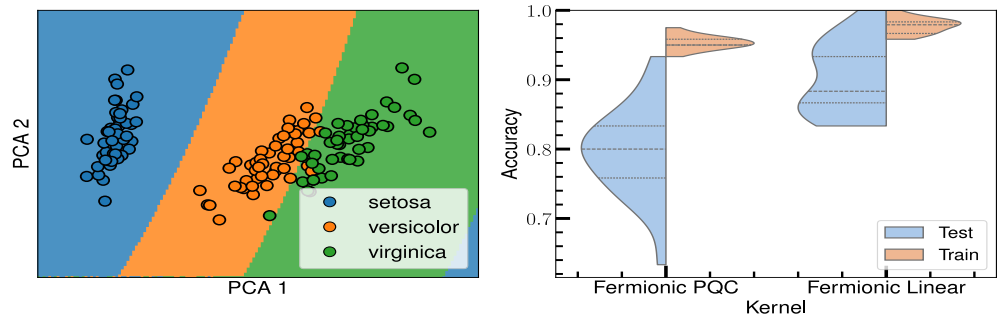


Figure 3: Output of the machine learning example code. On the left: Example of a decision boundaries for the Fermionic Linear classifier on the iris dataset. On the right: Violin plots showing the distribution of training and testing accuracies across different cross-validation folds for both the Fermionic PQC and Fermionic Linear classifiers.

Acknowledgements

The author thanks Victor Drouin-Touchette for comments on the main text and Stefanos Kourtis for his support on the topic. This research was funded by the research Chair in Quantum Computing funded by Ministère de l'Économie, de l'Innovation et de l'Énergie, the QSciTech CREATE program funded by the Natural Sciences and Engineering Research Council of Canada (NSERC), and NSERC Discovery and Alliance grants.

AI usage disclosure

The author acknowledges the use of AI-assisted tools during the preparation of this manuscript to improve grammar, style, and clarity of presentation. AI tools were also used to assist in drafting limited portions of the source code documentation for the purpose of improving readability. No scientific content, results, or conclusions were generated by AI tools. The author has reviewed, edited, and validated all AI-assisted content and assumes full responsibility for the final version of the manuscript and the software.

References

- Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., ... Chintala, S. (2024, April). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. <https://doi.org/10.1145/3620665.3640366>
- Bergholm, V., & et.al. (2022). *PennyLane: Automatic differentiation of hybrid quantum-classical computations*. <https://arxiv.org/abs/1811.04968>
- Biamonte, J., & Bergholm, V. (2017). Tensor networks in a nutshell. *arXiv Preprint arXiv:1708.00006*.
- Bravyi, S. (2008). *Contraction of matchgate tensor networks on non-planar graphs* (No. arXiv:0801.2989). arXiv. <http://arxiv.org/abs/0801.2989>
- Brod, D. J. (2016). Efficient classical simulation of matchgate circuits with generalized inputs and measurements. *Physical Review A*, 93(6), 062332. <https://doi.org/10.1103/PhysRevA.93.062332>
- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.
- Chapman, A., & Miyake, A. (2018). Classical simulation of quantum circuits by dynamical localization: Analytic results for pauli-observable scrambling in time-dependent disorder. *Phys. Rev. A*, 98, 012309. <https://doi.org/10.1103/PhysRevA.98.012309>
- Dai, L., Kuo, W., & Chung, M.-C. (2015). Extracting entangled qubits from Majorana fermions in quantum dot chains through the measurement of parity. *Scientific Reports*, 5(1), 11188. <https://doi.org/10.1038/srep11188>
- Fisher, R. A. (1936). *Iris*. UCI Machine Learning Repository.
- Gidney, C. (2021). Stim: A fast stabilizer circuit simulator. *Quantum*, 5, 497. <https://doi.org/10.22331/q-2021-07-06-497>

- 189 Gince, J., Pagé, J.-M., Armenta, M., Sarkar, A., & Kourtis, S. (2024). Fermionic machine
190 learning. *2024 IEEE International Conference on Quantum Computing and Engineering*
191 *(QCE)*, 01, 1672–1678. <https://doi.org/10.1109/QCE60285.2024.00195>
- 192 Gray, J. (2018). Quimb: A python package for quantum information and many-body
193 calculations. *Journal of Open Source Software*, 3(29), 819.
- 194 Hebenstreit, M., Jozsa, R., Kraus, B., & Strelchuk, S. (2020). Computational power of
195 matchgates with supplementary resources. *Phys. Rev. A*, 102, 052604. <https://doi.org/10.1103/PhysRevA.102.052604>
196
- 197 Jordan, P., & Wigner, E. (1928). Über das Paulische Äquivalenzverbot. *Zeitschrift für Physik*,
198 47(9), 631–651. <https://doi.org/10.1007/BF01331938>
- 199 Jozsa, R., & Miyake, A. (2008). Matchgates and classical simulation of quantum circuits.
200 *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*,
201 464(2100), 3089–3106. <https://doi.org/10.1098/rspa.2008.0189>
- 202 Laar, M. J. van. (2022). *Quantum aspects of variational quantum circuit-based models* [PhD
203 thesis].
- 204 Mocherla, A., Lao, L., & Browne, D. E. (2023). Extending matchgate simulation methods to
205 universal quantum circuits. *arXiv Preprint arXiv:2302.02654*.
- 206 Nielsen, M. A., & others. (2005). The fermionic canonical commutation relations and the
207 jordan-wigner transform. *School of Physical Sciences The University of Queensland*, 59,
208 75.
- 209 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,
210 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,
211 Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python.
212 *Journal of Machine Learning Research*, 12, 2825–2830.
- 213 Projansky, A. M., Necaie, J., & Whitfield, J. D. (2025). Gaussianity and simulability of
214 cliffords and matchgates. *Journal of Physics A: Mathematical and Theoretical*, 58(19),
215 195302.
- 216 Valiant, L. G. (2001). Quantum computers that can be simulated classically in polynomial
217 time. *Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing*,
218 114–123. <https://doi.org/10.1145/380752.380785>
- 219 Wick, G. C. (1950). The evaluation of the collision matrix. *Physical Review*, 80(2), 268–272.
220 <https://doi.org/10.1103/PhysRev.80.268>